MDMC
Master in Data Management and Curation

AREA SCIENCE PARK

SISSA

Python II: APIs and HTTP

Jacopo Salvalaggio



Image credit: pngfre.com

CC BY

Goals

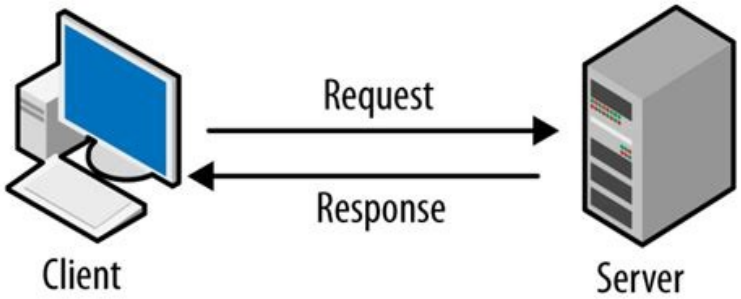


Image credit: ar.inspiredpencil.com

Objectives of the lecture:

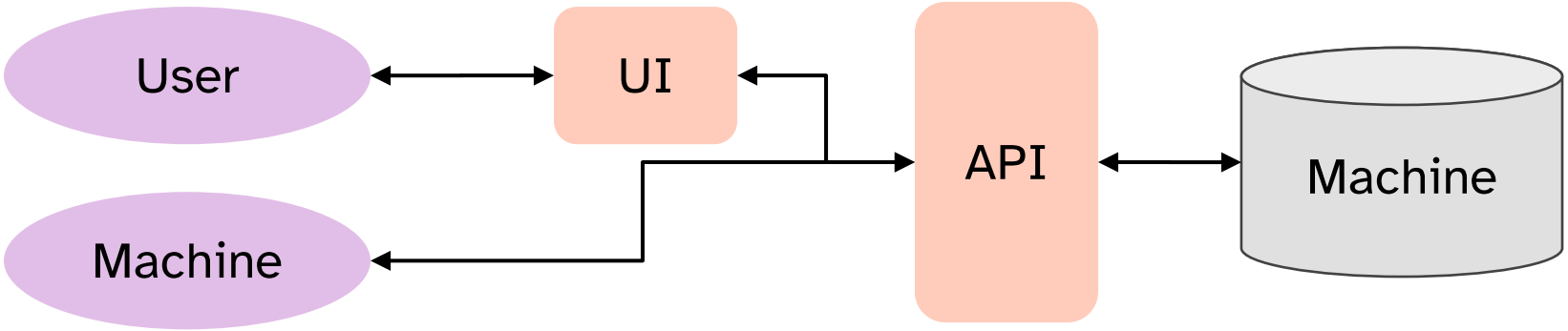
* Understand the meaning of (REST) **APIs**
* Define the basics of **HTTP**
* Write a simple **HTTP API** in Python with **Flask**

Main sources

* [1] HTTP Semantics: https://httpwg.org/specs/rfc9110.html
* [2] Fielding, R., “Architectural Styles and the Design of Network-based Software Architectures”, Doctoral Dissertation, University of California, Irvine, September 2000, <https://roy.gbiv.com/pubs/dissertation/top.htm>
* [3] Flask "Quickstart": https://flask.palletsprojects.com/en/stable/quickstart/

API

API (Application Programming Interface) is the set of tools and procedures used to **communicate** with a **machine** or more in general with **code**.

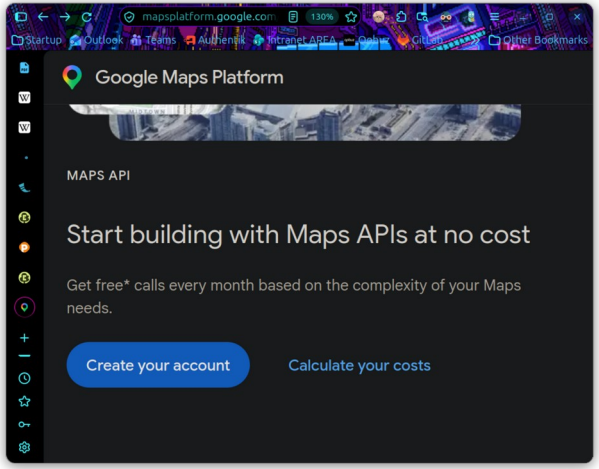


An examPle (Interesting)

An **API** can present itself in **many forms**. For instance, it can be a set of functions of a Python package that the user does usually not interface with directly.

Example: UI and API in a Python package

```
>>> import improc
>>> pic = improc.JPGImage()
>>> pic.load("filename") # UI
>>> pic._load_array(arr) # API
```



Client-server architecture

Often the target machine is a **server** containing some data we (the **client**) need to access. The API allows sending of **requests** from the cliend and receiving of **responses** from the server.

This is the principle of **"separation of concerns"**.

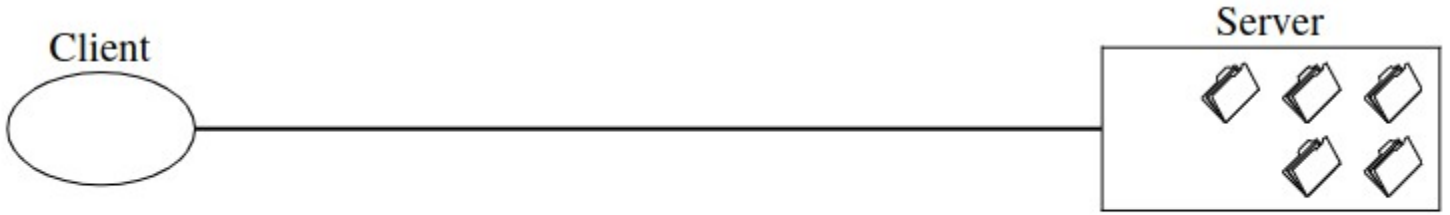


Figure 5-2. Client-Server

HTTP

HTTP (HyperText Transfer Protocol) is the most widely used protocol to **handle the communication** between a client and a server.

[HTTP] began as a trivial mechanism for low-latency requests, with a single method (GET) to request transfer of a presumed hypertext document identified by a given pathname. [1]



REST

The principles behind HTTP are the ones of the **REST** (Representational State Transfer) style.

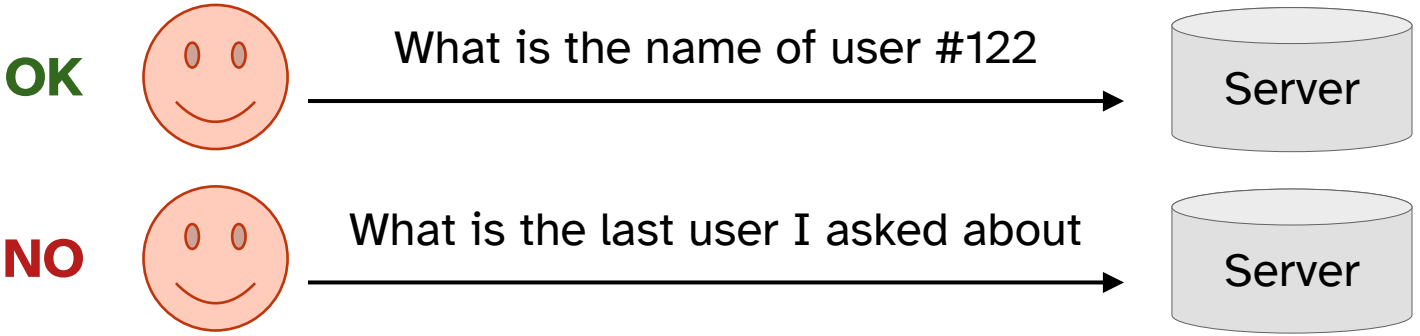
A RESTful communication is:

* **stateless** (server-side)
* **cacheable**
* targeting a **resource** through an **identifier**
* manipulating or transferring its **representation**

"Stateless"

The communication must be **stateless** on the side of the **server**, meaning that all information about the request should be provided by the client in each interaction.

No information about the "session" is stored on the server.



"Cacheable"

Some responses by the server may be saved by the client in a **cache**, in order to speed up future communications.

The communication must hold information on what can be cacheable: **some things** vary through time and **need to be updated** continuously.

Again, the statefulness here is always client-side, never the server will cache data.

See also: cookies

"Resource"



A **resource** is the abstract idea of any piece of information.

We address the resource via its identifier, the **URI** (Uniform Resource Identifier) (for web-specific applications **URL** - Uniform Resource Locator)

Example: https://httpwg.org/specs/rfc9110.html is the URI for the resouce "official HTTP semantics documentation".

Image credits: philosophyalevel.com

"Representation"

A **representation** is the concrete form of a resource. In the REST paradigm its format should be standard.

The standard representation for a web page resource is an HTML file (nowadays HTML + CSS + JS usually).

Example: https://httpwg.org/specs/rfc9110.html is a representation in HTML form.

Too abstract?

All of this sounds **quite abstract**! But in the end it is just telling us that HTTP is a stateless protocol to make requests and receive websites and other file formats (representations) in return.



HTTP request

An HTTP request is the one made by the client to the server. It can entail different actions specified by the **method**:

* **GET**: get representation of the target resource
* **POST**: have the target resource process the request data
* **PUT**: set the representation of a given resource to be a certain value
* **DELETE**: remove representations of target resource

HTTP response

The server will reply to the client with a response (e.g. returning the representation of a resource) and a **status code**:

* **200**: OK, the request has succeded
* **404**: Not Found, the server could not find the representation for the target resource
* **500**: Internal Server Error, the server encountered a problem and could not handle the request

APIs over HTTP

A code can implement an HTTP RESTful API to access or modify the representation of its associated resources.

You can pass arguments to the resource you are accessing by adding data to a POST request or adding a **query** to the URI (called also **endpoint**) of a GET* request with the syntax

`uri?name=value&other_name=other_value&...`

Example resource: weather in a given city → the API request will be a GET to `weather.com?city=trieste`

HTTP requests in bash

The standard library for HTTP request in bash is **`curl`**.

Example: curl

```
$ curl -X GET www.google.com/jacopo
<!DOCTYPE html>
  [...]
  <title>Error 404 (Not Found)!!1</title>
  [...]
  <p><b>404.</b> <ins>That's an error.</ins>
  <p>The requested URL <code>/jacopo</code> was not found on this
server.  <ins>That's all we know.</ins>
$ curl -X POST -d "some_data=some_value" www.somewhere.com/
```

Building your API with Python

You can set up an API server with Python via **Flask**

Example: Flask

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def hello_world():
    return "Hello, world!\n", 200

if __name__ == "__main__":
    app.run()
```

Passing variable values

You can treat a piece of the **endpoint** as a **variable** or pass data in the **query**. Note: default method is GET.

Example: passing variables

```
from flask import Flask, request
from markupsafe import escape
app = Flask(__name__)
@app.route("/user/<user_id>")
def hello_world(user_id):
    values = {"1": 1, "ciao": 2}
    value = values[user_id]
    return value, 200
@app.route("/greet")
def greet():
    name = request.args.get("name")
    return f"Hi, {escape(name)}!", 200
```

GET and POST

You can also pass data as a **POST form**.

Example: POST

```
from flask import Flask, request
from markupsafe import escape
app = Flask(__name__)
@app.route("/greet", methods=["POST"])
def greet():
    name = request.form["name"]
    return f"Hi, {escape(name)}!", 200
```

You can run `curl -X POST -d "name=Jacopo" <URI>`

Last slide ever!



Thanks!



AREA SCIENCE PARK

SISSA

OGS Istituto Nazionale di Oceanografia e di Geofisica Sperimentale

ARCTUR

REGIONE AUTONOMA FRIULI VENEZIA GIULIA

E-ARGO



Image credit: Abel+Imray

Thank you for your attention :)

Image credits for background rainbow gradient: Image Colorful Rainbow Pattern by sgagnonou812 at Wallpapers.com